



JPA - FetchType: LAZY vs EAGER

Cómo y cuándo se cargan las relaciones en JPA

Una guía práctica para desarrolladores Java que buscan optimizar el rendimiento de sus aplicaciones mediante el control estratégico de la carga de datos.

- ❏ Introducir el tema recordando que ya vimos cómo mapear relaciones, y ahora veremos cómo se comportan al consultarlas.

Recordatorio: Nuestros Proyectos Anteriores

Relación OneToOne

Persona ↔ Domicilio



Una persona tiene exactamente un domicilio, y cada domicilio pertenece a una única persona. Esta relación bidireccional nos permite navegar en ambas direcciones.

Relación ManyToMany

Persona ↔ Curso



Una persona puede inscribirse en múltiples cursos, y cada curso puede tener múltiples estudiantes inscritos. Requiere una tabla intermedia para manejar la asociación.

Ahora vamos a observar **qué sucede exactamente** cuando traemos estas entidades desde la base de datos y cómo podemos controlar este comportamiento.

📌 Explicar que ahora vamos a observar qué pasa al traer estas entidades desde la base de datos.

¿Qué es FetchType?

FetchType es la **estrategia que define cómo JPA carga las relaciones** asociadas a una entidad. Es una decisión crítica que afecta tanto el rendimiento como el comportamiento de tu aplicación.

- 1 — FetchType.EAGER
Carga **todo junto** inmediatamente
- 2 — FetchType.LAZY
Carga **bajo demanda** cuando se necesita

Esta decisión determina cuándo se ejecutan las consultas SQL y qué datos se transfieren desde la base de datos. La elección incorrecta puede resultar en problemas de rendimiento graves.

📌 Conectar con la idea de "cargar todo ya" vs "cargar bajo demanda".



FetchType.EAGER: Carga Inmediata

Características

- Carga automática junto con la entidad principal
- Una sola consulta SQL (JOIN)
- Datos disponibles inmediatamente
- Mayor consumo de memoria

Ejemplo Práctico

```
@Entity
public class Persona {
    @OneToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "domicilio_id")
    private Domicilio domicilio;
}

// Al ejecutar:
Persona persona = em.find(Persona.class, 1L);
// El domicilio YA está cargado automáticamente
System.out.println(persona.getDomicilio().getCalle());
```

Ventaja: Simplicidad total - no hay sorpresas ni excepciones.

Riesgo: Consultas más pesadas y posible sobrecarga de datos innecesarios.

📌 Ventaja = simplicidad, riesgo = consultas más pesadas.

FetchType.LAZY: Carga Bajo Demanda

01

Consulta inicial

Solo se carga la entidad principal, las relaciones quedan como **proxies**

02

Acceso explícito

Cuando accedes a la relación, JPA ejecuta una consulta adicional automáticamente

03

Datos disponibles

La información se carga y queda disponible para el resto de la sesión

```
@Entity
public class Persona {
    @ManyToMany(fetch = FetchType.LAZY) // Por defecto
    private List<Curso> cursos;
}

// Al ejecutar:
Persona persona = em.find(Persona.class, 1L); // Solo carga Persona
// Los cursos NO están cargados todavía
List<Curso> cursos = persona.getCursos(); // AHORA se ejecuta la consulta
```

Ventaja: Eficiencia - solo carga lo que realmente necesitas.

Riesgo: LazyInitializationException si accedes fuera del contexto de persistencia.

📌 Ventaja = eficiencia, riesgo = LazyInitializationException si accedemos fuera del contexto.

Ejemplo Práctico: OneToOne Persona-Domicilio

Con FetchType.EAGER

```
@Entity
public class Persona {
    @OneToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "domicilio_id")
    private Domicilio domicilio;
}
```

```
// SQL generado:
SELECT p.*, d.*
FROM persona p
LEFT JOIN domicilio d
    ON p.domicilio_id = d.id
WHERE p.id = ?
```

✓ Una sola consulta

⚠ Siempre trae el domicilio

Con FetchType.LAZY

```
@Entity
public class Persona {
    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "domicilio_id")
    private Domicilio domicilio;
}
```

```
// SQL generado (primera consulta):
SELECT p.*
FROM persona p
WHERE p.id = ?
```

```
// SQL adicional (al acceder):
SELECT d.*
FROM domicilio d
WHERE d.id = ?
```

✓ Carga selectiva

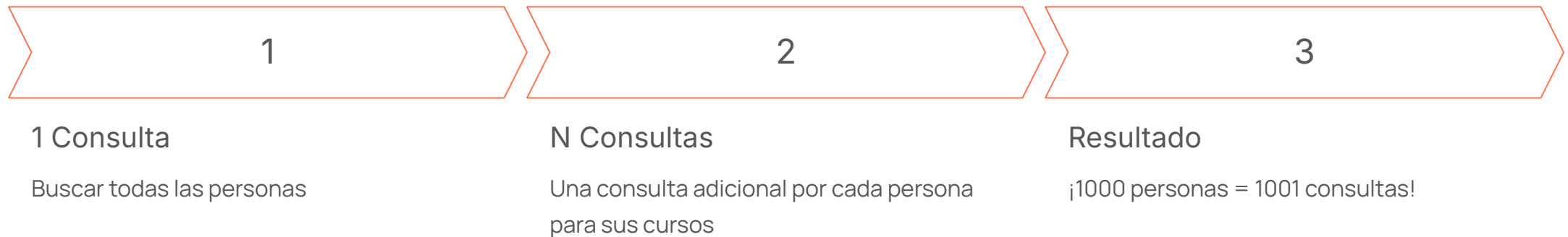
⚠ Dos consultas si necesitas el domicilio

📄 Comparar el SQL generado en cada caso.

Ejemplo Práctico: ManyToMany Persona-Curso

El Problema del N+1 con EAGER

```
@Entity
public class Persona {
    @ManyToMany(fetch = FetchType.EAGER) // ¡Peligroso para colecciones!
    @JoinTable(
        name = "persona_curso",
        joinColumns = @JoinColumn(name = "persona_id"),
        inverseJoinColumns = @JoinColumn(name = "curso_id")
    )
    private List<Curso> cursos;
}
```



Recomendación: LAZY es la opción por defecto para colecciones (@OneToMany, @ManyToMany) precisamente por este motivo. Si necesitas cargar la colección, usa JOIN FETCH o Entity Graphs estratégicamente.

📌 Contar que LAZY es la mejor opción para colecciones grandes.

Buenas Prácticas y Recomendaciones



LAZY por Defecto

Usa FetchType.LAZY para colecciones (@OneToMany, @ManyToMany). Es más eficiente y evita el problema N+1.



EAGER Selectivo

Solo usa EAGER en relaciones @OneToOne o @ManyToOne cuando estés seguro de que siempre necesitarás esos datos.



JOIN FETCH

Para cargas específicas, usa consultas JPQL con JOIN FETCH en lugar de cambiar el fetch type global.



Monitoreo SQL

Siempre revisa las consultas SQL generadas. Activa show-sql=true durante el desarrollo.

Ejemplo de Carga Controlada

```
// En lugar de cambiar fetch type globalmente:  
@Query("SELECT p FROM Persona p JOIN FETCH p.cursos WHERE p.id = :id")  
Persona findPersonaWithCursos(@Param("id") Long id);  
  
// O usando Entity Graphs (JPA 2.1+):  
@EntityGraph(attributePaths = {"cursos"})  
Persona findById(Long id);
```

📌 Conectar con problemas comunes que se ven en la práctica.

DEMO: FetchType en Acción

Demostración en Vivo

Vamos a ver el comportamiento de FetchType en tiempo real con nuestros ejemplos de Persona-Domicilio y Persona-Curso.

1 Persona-Domicilio con EAGER

Mostraremos cómo se carga automáticamente el domicilio y analizaremos el SQL generado con una sola consulta JOIN.

2 Persona-Curso con LAZY

Veremos cómo los cursos no se cargan inicialmente y se requiere acceso explícito para desencadenar la consulta adicional.

3 Problema N+1 en Acción

Forzaremos EAGER en ManyToMany y observaremos cómo se multiplican las consultas SQL innecesariamente.

Nota importante: El código completo se ejecutará en vivo para que puedas ver el comportamiento real de JPA, incluyendo los logs de SQL y los tiempos de ejecución.

 El código se mostrará en vivo, no en la diapositiva.

Resumen y Próximos Pasos

Diferencias Clave: LAZY vs EAGER

FetchType.LAZY

- Carga bajo demanda
- Más eficiente por defecto
- Ideal para colecciones
- Requiere contexto activo

FetchType.EAGER

- Carga inmediata
- Más simple de usar
- Solo para relaciones pequeñas
- Puede causar problema N+1

Ejercicio Práctico Recomendado

01

Experimenta en tus proyectos

Modifica los FetchType en tus entidades
Persona-Domicilio y Persona-Curso

02

Observa las consultas SQL

Activa show-sql=true y analiza cuántas
consultas se ejecutan en cada escenario

03

Mide el rendimiento

Compara tiempos de ejecución con diferentes
estrategias de carga

Recuerda: La optimización prematura es la raíz de todos los males, pero entender cómo funciona JPA te permitirá tomar decisiones informadas cuando el rendimiento realmente importe.

 Cierre motivando a experimentar y mirar siempre las consultas que produce JPA.